

JAVA INTERVIEW PREPARATION

CHAPTER 60

MEMORY LEAKS AND
OUT-OF-MEMORY DIAGNOSIS

Senior / Lead Java Developer Textbook Edition

Full reading material - not summarized

Diagnosing Memory Growth, Leaks, and OutOfMemoryError

Senior / Lead Java Developer Reading Chapter

A Java memory incident is not automatically a heap leak. The process uses heap, metaspace, code cache, thread stacks, direct buffers, memory-mapped files, JVM native structures, native libraries, and operating-system page cache. A container can be killed for exceeding its memory limit without the JVM throwing `OutOfMemoryError` or writing a heap dump.

The investigation must answer two questions separately:

1. Which memory area is growing or exhausted? 2. Why is that memory still needed, or why is allocation demand greater than capacity?

```
process memory growth
|-> Java heap: live objects or allocation pressure
|-> JVM native: metaspace, code cache, GC, symbols, threads
|-> application native: direct buffers, JNI, mapped files
|-> container/OS accounting and other processes
```

Stabilize and preserve failure evidence

Memory incidents often develop slowly and disappear after restart. Before restarting every affected instance, preserve evidence from at least one representative process if customer impact and safety permit.

Capture:

- exact `OutOfMemoryError` detail message or container termination reason;
- heap used before and after GC, allocation and promotion rate;
- process resident set size and container working set;
- class count, thread count, direct-buffer metrics, and metaspace;
- GC logs or JFR for the growth interval;
- class histograms at separated times;
- heap dump when heap retention is suspected;
- Native Memory Tracking data if it was enabled at startup.

Heap dumps can be as large as the heap and can pause or intensify I/O and disk pressure. Verify free disk space, choose an approved secure path, and restrict access because the dump may contain credentials, personal data, tokens, and business payloads.

For future incidents, a production policy commonly includes a tested dump path and `-XX:+HeapDumpOnOutOfMemoryError`, with external collection and encryption. The flag has little ongoing cost, but dump creation at failure still requires storage and time.

Read the exact failure signal

`OutOfMemoryError` is a family of allocation failures. The detail message narrows the search.

Java heap space means a heap allocation could not be satisfied. The heap may be undersized, the live set may have grown, an unexpectedly large object may be requested, or references may be retained accidentally.

GC overhead limit exceeded indicates that excessive time is being spent collecting with little memory recovered. Disabling the safeguard does not fix the cause.

Metaspace points toward class metadata: unbounded dynamic class generation, class-loader retention, or a configured limit that is too low.

Direct buffer memory indicates pressure from off-heap NIO buffers or a direct-memory limit. Heap analysis may reveal the small Java wrapper objects even though the large bytes live outside the heap.

Unable to create native thread means the JVM could not create another operating-system thread, possibly because of process limits, virtual memory for stacks, container memory, or runaway platform-thread creation.

Requested array size exceeds VM limit points to a single invalid or overflowed array request, not gradual leakage.

Native allocation failures or operating-system kills may have different messages or no Java exception. Inspect container status, kernel events, fatal error files, and platform metrics.

Leak, pressure, and capacity are different

A leak is memory retained beyond its useful lifetime because a reachable reference still exists. Allocation pressure means objects are created rapidly but may die normally. Capacity shortage means the legitimate live set and workload do not fit the configured memory.

```
after-GC heap stable + high allocation -> churn/pressure
after-GC heap rises over time          -> growing live set
after-GC heap jumps with workload      -> capacity or unbounded request
RSS rises while heap stable            -> investigate native/off-heap/accounting
```

Do not label every rising heap chart a leak. Caches warm, JIT compiles, classes load, and traffic state accumulates before reaching a plateau. The key evidence is whether memory expected to become unreachable is reclaimed after representative lifecycle completion and GC.

Use a time series, not one snapshot

Compare memory after collection over time. The sawtooth shape of normal allocation rises between collections and falls afterward. A leak often raises the floor.

```
normal:  /| /| /| /|      floor remains similar
         / / / /
leak:    /| /| /| /|      floor trends upward
         /|_ /|_ /|_ /|_
```

Correlate the trend with request rate, tenant count, cache entries, open sessions, class count, thread count, deployed version, and scheduled jobs. A daily rise can follow a report, refresh, or rollover rather than continuous traffic.

Heap occupancy percentages are misleading when the JVM changes committed heap size. Use bytes and collector phase. Process RSS can remain high after Java objects die because allocators and the JVM do not necessarily return pages immediately to the operating system.

Class histograms: identify growers

A class histogram shows instance count and shallow bytes by class. Collecting histograms at separated times can reveal growing types.

```
jcmd <pid> GC.class_histogram filename=histo-1.txt
... wait through representative workload ...
jcmd <pid> GC.class_histogram filename=histo-2.txt
```

The command can have impact because it may request heap traversal or collection depending on options and JVM behavior. Use it deliberately on a representative instance.

A large number of `byte[]`, `String`, or `HashMap$Node` instances is only a clue. These ubiquitous classes belong to many owners. A histogram reports shallow storage, not the retaining path. The next step is usually a heap dump and dominator analysis.

Compare both instance count and bytes. One large array and millions of tiny map nodes represent different problems.

Heap dumps and retained size

A heap dump captures objects and references. Analysis tools compute dominators and retained size: the amount of memory that could become collectible if a particular object were removed from the reachability graph.

```
GC root
-> ApplicationContext
  -> CacheManager
    -> map
      -> millions of response objects
```

The response objects may dominate the heap, but the cache manager or map explains why they remain reachable. Look for a small number of owners retaining a large subgraph.

Shallow size is the object's own storage. Retained size includes objects kept alive exclusively through it. A tiny collection object can retain gigabytes. Dominator reports are estimates based on graph reachability and shared ownership; inspect actual paths to GC roots before changing code.

A repeatable heap-dump comparison method

One dump near failure can identify a dominant owner, but two lifecycle-aligned dumps are stronger. Capture the first after warm-up and a full representative business cycle, then a second after repeated cycles and expected expiry. Keep traffic shape and configuration recorded.

Use this sequence:

1. Compare total live objects and dominant retained sets.
2. Identify classes whose instance count or retained bytes grew materially.
3. Group duplicate strings or payloads only as clues, not owners.
4. Select growing objects and inspect shortest and all meaningful paths to GC roots.
5. Move upward to the application component that owns the retaining collection, thread, loader, or task.
6. Confirm that the owner's size correlates with a measurable resource such as cache entries, queue depth, tenants, or unfinished requests.
7. Reproduce the lifecycle in a controlled test and verify reclamation after the fix.

Heap dumps from different JVM moments cannot always be mechanically subtracted because object identities differ. Compare aggregate class and owner behavior. A dominator that is large in both dumps may be legitimate baseline state; growth and lifetime semantics determine suspicion.

Leak-suspect reports require judgment

Analysis tools may label a dominant object as a leak suspect because it retains a large fraction of the heap. A correctly sized cache, compiled application metadata, or a deliberately loaded dataset can do the same. The tool does not know the business lifetime.

Ask whether the retained population is bounded by configuration, whether entries expire, whether the key cardinality matches the contract, and whether memory returns after the owning lifecycle ends. Treat automated reports as navigation aids.

Conversely, a real leak may be distributed across many owners and not appear as one dramatic dominator. Per-tenant maps, repeated class loaders, or thousands of small thread-local values can require grouping by class loader, package, allocation site, or owner type.

GC roots and reference paths

GC roots include active thread stacks, static fields, JNI references, system class loaders, and JVM internal structures. An object is collectible only when no strong path from a root remains.

Typical paths reveal the defect:

- static collection -> entries never removed;
- thread -> `ThreadLocalMap` -> value retained by pooled thread;
- class loader -> loaded class -> static state -> application objects;
- listener registry -> subscriber no longer used;
- executor queue -> tasks and captured request payloads;
- cache -> keys with unbounded cardinality;
- session registry -> expired sessions not purged.

Do not remove a reference merely because it is a GC-root path. Determine ownership and intended lifetime. The correction may be explicit unregistering, bounded eviction, task cancellation, or lifecycle closure.

Common leak: unbounded caches and maps

A map is not a cache until it has a capacity and eviction policy. TTL alone may not bound memory during a traffic burst, and expiry may be lazy.

```
// Unbounded cardinality: every distinct query remains forever.
private final ConcurrentMap<String, SearchResult> results =
    new ConcurrentHashMap<>();

SearchResult find(String query) {
    return results.computeIfAbsent(query, searchService::execute);
}
```

Define maximum weight or entry count, expiry, key normalization, negative-cache behavior, and observability. The value's true weight may include nested graphs and external buffers, not only the top object.

Cache stampedes can create transient memory explosions even when steady-state capacity is bounded. Coalesce loaders and limit in-flight population.

Common leak: ThreadLocal values

Thread-local state attached to a long-lived pool thread can outlive the request. Even when a `ThreadLocal` key becomes weakly reachable, its value may remain until map cleanup occurs.

```
try {
    requestContext.set(context);
    chain.doFilter(request, response);
} finally {
    requestContext.remove();
}
```

Framework-managed context propagation is safer than ad hoc thread locals, especially with asynchronous execution. A context captured into queued tasks can also retain request graphs after the request completes.

Virtual threads have different lifecycle economics because they are normally short-lived, but values can still increase per-task footprint and captured state can remain reachable through task owners.

Common leak: executor queues and incomplete work

An unbounded executor queue retains every submitted `Runnable`, its lambda captures, input payload, tracing context, and future. If producers outrun consumers, memory grows as deferred work.

```
arrival 2,000 tasks/s
completion 1,500 tasks/s
queue grows 500 tasks/s plus captured object graphs
```

Use bounded queues, rejection or backpressure, cancellation, deadlines, and metrics for queue depth and task age. A bigger heap only gives the queue more time to grow.

`CompletableFuture` graphs can retain inputs, callbacks, and exceptional results until the graph completes and references are released. Timeouts must cancel or detach underlying work where supported; completing the caller's timeout future does not always stop the original operation.

Common leak: listeners, registries, and class loaders

Observers and callbacks create reverse ownership: a long-lived publisher retains subscribers. Components must unregister during shutdown or use an ownership model that removes dead entries safely.

Class-loader leaks appear in redeployment, plugin systems, dynamic scripting, proxies, and bytecode generation. A parent or long-lived thread may retain an old application class loader through context class loader, static registry, driver manager, MBean, thread local, or library cache. Then every class and static field loaded by it remains reachable.

Monitor loaded and unloaded class counts and metaspace across deployments. Heap dumps can show multiple generations of similarly named classes under different loaders. Raising metaspace postpones failure but does not repair loader retention.

Direct buffers and off-heap memory

Direct buffers reduce copying for native I/O but allocate memory outside the Java heap. Their Java wrapper and cleanup state remain on heap. If wrappers are retained, native memory remains. Cleanup timing can also lag under unusual allocation and GC behavior.

Track direct-buffer pool count and bytes where available. Libraries such as networking frameworks may pool buffers intentionally; a high stable pool is not automatically a leak. Check reference-count ownership, release paths, arena limits, and workload correlation.

Memory-mapped files can increase RSS or virtual size and are accounted differently by the OS. Distinguish anonymous private memory from mapped file pages before concluding that the JVM leaked native heap.

Native Memory Tracking

Native Memory Tracking (NMT) categorizes memory allocated internally by HotSpot. It must be enabled when the JVM starts, commonly at summary or detail level.

```
-XX:NativeMemoryTracking=summary
jcmd <pid> VM.native_memory baseline
... later ...
jcmd <pid> VM.native_memory summary.diff
```

Categories include Java heap reservation and commitment, classes, threads, code, GC, compiler, and internal structures. Reserved memory is address space; committed memory is backed for use. Do not add reserved values and compare them directly with RSS.

NMT does not track every allocation made by arbitrary JNI or third-party native code. If RSS grows while heap and NMT categories remain stable, investigate native libraries, mapped files, allocator behavior, and OS accounting with platform tools.

Thread count and native stacks

Every platform thread consumes native resources and stack address space. An unbounded thread-per-request design can fail with `unable to create native thread` before heap is full.

Track live, daemon, and peak thread count, thread creation rate, and executor ownership. Thread dumps reveal repeated names and creation sites may require JFR or source tracing. Check operating-system process limits and container memory.

Reducing `-Xss` can permit more threads but increases stack-overflow risk and does not fix runaway creation. Use bounded executors or virtual threads for appropriate blocking tasks while preserving downstream concurrency limits.

Container memory and OOM kills

The container limit normally covers more than Java heap. If `-Xmx` nearly equals the container limit, native JVM memory, stacks, direct buffers, and libraries have no headroom.

```
container limit
> max/committed heap
+ metaspace and code cache
+ thread stacks
+ GC and JVM native structures
+ direct/JNI memory
+ operational headroom
```

An external OOM kill can terminate the process abruptly, so heap-dump-on-OOM may never run. Monitor the platform termination reason and memory working set. Set resource limits from measured peak categories, not a fixed heap percentage copied between services.

Kubernetes restart loops and evidence retention

When a pod is killed and immediately replaced, the service may appear self-healing while losing every diagnostic artifact stored on its ephemeral filesystem. Preserve termination reason, previous-container logs, resource metrics, JFR, and dumps through an approved persistent or external collection path.

Readiness should prevent a newly started instance from receiving full traffic before caches, pools, and required state are ready. Otherwise an OOM restart can trigger a cold-start load surge, repeated failure, and a fleet-wide cascade.

Set restart policy and liveness behavior carefully. A liveness probe that kills a process during long GC pressure may shorten evidence collection and intensify churn. It is not a substitute for memory admission control and resource bounds.

During mitigation, keep at least one artifact-bearing failed generation while replacing enough capacity to recover. Automate artifact upload with size, encryption, retention, and access controls rather than depending on an operator to copy a multi-gigabyte dump during an outage.

Safe mitigation

Mitigation depends on the exhausted area. Reduce traffic or payload size, disable the leaking feature, bound a cache or queue, stop a producer, reduce thread creation, restart instances gradually, or increase a limit only when capacity shortage is established.

Avoid restarting the whole fleet simultaneously: cold caches and reloaded state can overload dependencies. Preserve one failing artifact, then replace instances gradually while watching memory slope.

Increasing heap can buy investigation time but also enlarge dumps and extend the time before recurrence. It is a capacity correction only when the live set is legitimate and the container has native headroom.

Verify the correction

Reproduce the lifecycle that leaked: repeated tenant creation and deletion, redeployment, failed requests, timeout storms, cache churn, file processing, or long soak load. A short load test may never reveal a slow leak.

Compare:

- after-GC live-set slope;
- class histogram deltas;
- retained owners in before/after dumps;
- process RSS and NMT category deltas;
- thread and direct-buffer counts;
- allocation rate and GC CPU;
- completed business operations and queue age.

The test should reach a plateau or return to baseline after work expires. Add bounds and alerts on the owning resource, not only a generic heap-percent alarm.

Common senior-level traps

"OutOfMemoryError means heap leak." Read the detail message and compare heap, native memory, thread, and container evidence.

"A large object type is the leak." The retaining owner and path to a GC root explain the lifetime.

"RSS stayed high after GC, so objects leaked." JVM and native allocators may retain committed pages; inspect heap and native categories.

Increase `Xmx`. This can delay a leak and remove native headroom.

"TTL makes a cache bounded." Entry arrival can exceed expiry and lazy cleanup may retain state.

"The timeout released the work." Underlying tasks and captured graphs may continue unless cancellation and ownership are explicit.

A strong interview answer

I first identify the exhausted memory area from the exact OOM or container signal and compare Java heap, after-GC live set, RSS, metaspace, direct buffers, thread count, and native categories. I preserve safe evidence before restart: time-series metrics, GC/JFR data, separated histograms, a protected heap dump for heap retention, and NMT diffs when enabled. In the heap dump I use dominators, retained size, and paths to GC roots to find the owner, not merely the largest class. I investigate unbounded caches, queues, thread locals, listeners, futures, class loaders, direct buffers, and thread creation. I distinguish a leak from allocation churn and legitimate capacity, mitigate without destroying all evidence, and verify the fix with a soak test that demonstrates a stable post-GC plateau and bounded resource counts.

Chapter summary

Memory diagnosis is ownership analysis across several allocators. Heap dumps explain reachable Java graphs; NMT and platform tools explain other process memory; time series establish whether growth is persistent. The correct fix bounds or releases the owner whose lifetime exceeds the business work.

Revision Notes

- Java process memory includes heap, JVM native areas, stacks, direct/JNI memory, and mappings.
- A container OOM kill may occur without Java throwing `OutOfMemoryError`.
- Read the exact OOM detail message before choosing tools.
- Distinguish leaks, allocation pressure, and legitimate capacity shortage.
- A rising post-GC floor is stronger leak evidence than a heap sawtooth.
- Compare bytes, not only percentages, and correlate with workload lifecycle.
- Histograms identify growing classes; they do not show retaining ownership.
- Retained size and paths to GC roots reveal why objects remain reachable.
- Common owners include caches, thread locals, executor queues, futures, listeners, and class loaders.
- TTL alone does not guarantee bounded cache memory.
- Bounded queues prevent deferred work from becoming a memory leak.
- Metaspace growth often requires class-loader and generated-class analysis.
- Direct buffers consume native memory even though wrappers are on heap.
- NMT must be enabled at startup and does not cover every external native allocation.
- Reserved native memory differs from committed memory and RSS.
- Platform threads consume native stacks and OS resources.
- `Xmx` must leave container headroom for nonheap memory.
- Heap dumps contain sensitive data and require secure handling and adequate disk.
- Verify fixes with lifecycle-specific soak tests and post-GC plateau evidence.